Lecture 15 — Dynamic Programming II: Knapsack, LIS, LCS, Edit Distance, Grid DP

> Year Curriculum · Prerequisites: Week 5 (binary search), Week 14 (DP I: memoization, tabulation, 1D DP) · Estimated time: 2.5–3 hours

1. Overview

In Week 14 we met dynamic programming with a single index: `dp[i]` = the answer for the first `i` elements, or for the value `i`. Most interesting DP problems need a state with **two coordinates**. This lecture covers the five classical families that every contestant must be able to write from memory:

1. **Knapsack** — choose a subset of items under a capacity constraint.
2. **Longest Increasing Subsequence (LIS)** — both the $O(n^2)$ DP and the $O(n \log n)$ "tails" method.
3. **Longest Common Subsequence (LCS)** — the prototypical "prefix of one sequence × prefix of another" DP.
4. **Edit distance** — the same state shape with three transitions.
5. **Grid DP** — counting or optimising paths that move only right and down.

Beyond the specific algorithms, the goal is to recognise the *state shapes*. Contest problems rarely say "this is knapsack"; they describe shopping, packing, splitting a set into two halves, scheduling jobs — and you must see the "items × budget" state underneath.

Throughout, keep the five-step recipe from Week 14 in front of you: **state, transition, base cases, evaluation order, answer**, and estimate complexity as *number of states × cost per transition*.

2. 0/1 knapsack

2.1 Problem

We have `n` items; item `i` has weight (cost) `w[i]` and value `v[i]`. We have capacity `W`. Choose a subset with total weight at most `W` maximising total value. Each item may be taken at most once — hence "0/1".

CSES 1158 *Book Shop* is exactly this: books have prices and page counts, and we have `x` money.

2.2 Why greedy fails

Sorting by value/weight ratio is optimal for the *fractional* knapsack (where you may take half an item) but not here. Capacity 10, items (6, 7), (5, 5), (5, 5): greedy takes the 6-weight item first (ratio 1.17), after which neither 5-weight item fits, giving 7. Taking the two 5-weight items gives 10. Whenever an exchange argument (Week 4) breaks down like this, suspect DP.

2.3 The 2D formulation

State. `dp[i][c]` = maximum value obtainable using only items `0..i-1` with total weight at most `c`.

Transition. Item `i-1` is either skipped or taken:

```
dp[i][c] = max(dp[i-1][c], dp[i-1][c - w[i-1]] + v[i-1])   // second term only if c >= w[i-1]
```

Base. `dp[0][c] = 0` for every `c` (no items, no value).

Answer. `dp[n][W]`. Because the state says "weight *at most* c", the table is monotone in `c` and the answer is at the last column. (If you define the state as "weight *exactly* c", initialise unreachable states to $-\infty$ and take the max over the last row.)

Complexity. `(n+1)(W+1)` states, O(1) each: O(nW) time. For Book Shop, $n \cdot x = 1000 \cdot 10^5 = 10^8$ simple operations, which runs in well under a second with tight loops — but a 10^8-entry `int` table is 400 MB. We need to reduce memory.

2.4 Rolling the table

Row `i` reads only row `i-1`. So we keep a single array `dp[c]` and update it in place. The subtle point is the **direction** of the loop over `c`:

```
for (int i = 0; i < n; i++)
    for (int c = W; c >= w[i]; c--)
        dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
```

When we compute `dp[c]` going downward, the cell `dp[c - w[i]]` (a smaller index) has not yet been updated in this round, so it still holds the value from row `i-1`. That is exactly the 2D recurrence.

If we loop **upward**, `dp[c - w[i]]` has already been updated in this round — it may already include item `i` — so item `i` can be added again. That gives the **unbounded knapsack** (each item usable any number of times). It is a bug for 0/1 knapsack and the correct algorithm for unbounded knapsack. Memorise: *descending = once, ascending = unlimited*.

Trace. Items (2, 3) and (3, 4), capacity 5:

c	0	1	2	3	4	5
initial	0	0	0	0	0	0
after item (2,3)	0	0	3	3	3	3
after item (3,4)	0	0	3	4	4	7

Had we looped upward for the first item, we would get `dp[4] = dp[2] + 3 = 6` — the item used twice.

2.5 The knapsack family

The same skeleton solves a surprising number of problems:

- **Counting subsets with a given sum** (CSES 1093 *Two Sets II*): `dp[c] += dp[c - w]`, descending, modulo a prime.
- **Which sums are reachable** (CSES 1745 *Money Sums*): `dp[c] |= dp[c - w]`, descending. With `std::bitset` this becomes `reach |= reach << w`, a 64× speed-up.
- **Counting ordered vs. unordered coin combinations** (CSES 1635 and 1636): with coins in the *outer* loop you count unordered multisets; with the sum in the outer loop you count ordered sequences. This is a classic exam question — work out both on coins {1, 2}, sum 3 (answers 2 and 3).
- **Large capacity, small values** (AtCoder DP *Knapsack 2*, W up to 10^9 but total value ≤ 10^5): swap the roles. `dp[val]` = minimum weight to achieve exactly value `val`; the answer is the largest `val` with `dp[val] ≤ W`. The lesson: **index the DP by whichever quantity is small.**

3. Longest increasing subsequence

3.1 Definitions

A *subsequence* keeps the original order but may skip elements (unlike a *subarray*, which is contiguous). LIS asks for the longest strictly increasing subsequence. For `7 3 5 3 6 2 9 8` one LIS is `3 5 6 9`, length 4.

3.2 The O(n²) DP

State. `dp[i]` = length of the longest increasing subsequence that *ends exactly at* index `i`.

Why "ends at"? Because to extend a subsequence by `a[i]` we must know its last element, and fixing the last index fixes the last element.

```
dp[i] = 1 + max{ dp[j] : j < i and a[j] < a[i] }      (dp[i] = 1 if no such j)
answer = max_i dp[i]
```

$O(n^2)$ time — fine for n up to about 5000. This version is flexible: it adapts easily to weighted LIS, to "divisible chains" (CF 1350B *Orac and Models*, where `j` ranges over divisors of `i`), and to reconstructing the sequence via a parent array.

3.3 The O(n log n) "tails" algorithm

Maintain an array `tails`, where `tails[k]` is the **smallest possible last element** of any increasing subsequence of length `k+1` seen so far.

Invariant: `tails` is strictly increasing. (If a length-(k+2) subsequence ends with value `y`, its first k+1 elements form a length-(k+1) subsequence ending with something `< y`, so `tails[k] < tails[k+1]`.)

Processing a new element `a`:

- Find the first position `k` with `tails[k] >= a` using `lower_bound`.
- If there is none, `a` is larger than every tail; it extends the longest subsequence, so append it.
- Otherwise, the subsequence of length `k` (ending at `tails[k-1] < a`) can be extended by `a` to length `k+1` with a smaller tail than before; set `tails[k] = a`.

The answer is `tails.size()`.

element	**7**	**3**	**5**	**3**	**6**	**2**	**9**	**8**
tails	7	3	3 5	3 5	3 5 6	2 5 6	2 5 6 9	2 5 6 8

Note that the final `tails = 2 5 6 8` is **not** a subsequence of the input in order (2 comes after 6). Only its length is meaningful.

Strict vs. non-strict. For strictly increasing use `lower_bound` (an equal element replaces, it cannot extend). For non-decreasing use `upper_bound` (an equal element may extend). Getting this wrong is the single most common LIS bug.

Reconstruction. Store, alongside `tails`, the index `tailIdx[k]` of the element currently at `tails[k]`, and for each element `i` record `parent[i] = tailIdx[k-1]` at the moment it is placed at position `k`. Then follow parents from `tailIdx.back()`.

Using LIS as a subroutine. Many problems reduce to LIS after sorting. For example, "nested envelopes" (both dimensions strictly larger): sort by width ascending and, for equal widths, height *descending*, then take the LIS of heights. The descending tie-break prevents two envelopes of equal width from chaining.

4. Longest common subsequence

4.1 State

Given strings `s` (length n) and `t` (length m), find the longest string that is a subsequence of both.

State. `dp[i][j]` = LCS length of the prefixes `s[0..i)` and `t[0..j)`.

Transition. Look at the last characters `s[i-1]` and `t[j-1]`:

- If they are equal, it is always safe to match them: `dp[i][j] = dp[i-1][j-1] + 1`.
- Otherwise at least one of them is not in the LCS: `dp[i][j] = max(dp[i-1][j], dp[i][j-1])`.

Base. `dp[0][j] = dp[i][0] = 0`.

Order. Each cell depends on the cell above, to the left, and diagonally up-left; filling row by row, left to right, works.

O(nm) time and memory. Using `dp[i]` for the prefix of length `i` (so that index 0 is the empty prefix) avoids special-casing the borders, at the cost of writing `s[i-1]`. Be consistent.

4.2 Worked example

`s = axyb`, `t = abyxb`:

	ε	**a**	**b**	**y**	**x**	**b**
ε	0	0	0	0	0	0
a	0	1	1	1	1	1
x	0	1	1	1	2	2
y	0	1	1	2	2	2
b	0	1	2	2	2	3

The LCS has length 3 (`axb` and `ayb` both work).

4.3 Reconstruction

Start at `(n, m)`. If the characters match, that character is in the answer; move diagonally. Otherwise move to whichever neighbour (up or left) holds the value `dp[i][j]` — i.e. retrace the transition that produced it. Collect characters in reverse and reverse at the end. This "walk back through the transitions" technique works for every DP in this lecture; you just need to keep the full table rather than a rolled one.

5. Edit distance

The edit (Levenshtein) distance between `a` and `b` is the minimum number of single-character insertions, deletions and replacements turning `a` into `b` (CSES 1639).

State. `dp[i][j]` = edit distance between `a[0..i)` and `b[0..j)`.

Transition. Consider the last operation affecting the ends of the prefixes:

- delete `a[i-1]`: `dp[i-1][j] + 1`
- insert `b[j-1]` at the end: `dp[i][j-1] + 1`
- replace `a[i-1]` by `b[j-1]` (free if they are already equal): `dp[i-1][j-1] + [a[i-1] != b[j-1]]`

Base. `dp[i][0] = i` (delete everything), `dp[0][j] = j` (insert everything).

For `LOVE → MOVIE` the answer is 2: replace L by M and insert I. O(nm) time; with two rows, O(m) memory.

Notice how LCS and edit distance share the same state and the same dependency pattern. Whenever a problem involves two sequences and "aligning" them, start from `dp[i][j]` over prefix pairs. Variants include weighted edit costs, "minimum deletions to make two strings equal" (= n + m − 2·LCS), and the shortest common supersequence (= n + m − LCS).

6. Grid DP

CSES 1638 *Grid Paths*: an n×n grid has free cells `.` and traps `*`. Count the paths from the top-left to the bottom-right corner moving only right or down, modulo 10^9+7.

State. `dp[i][j]` = number of valid paths from (0, 0) to (i, j).

Transition. The last move came from above or from the left: `dp[i][j] = dp[i-1][j] + dp[i][j-1]`, or 0 on a trap.

Base. `dp[0][0] = 1` unless the start is itself a trap (a famous hidden test case).

Order. Since moves only go right and down, row-major order processes every cell after its predecessors. Formally, the cells with right/down edges form a *directed acyclic graph* and row-major order is a topological order — we will make this connection precise in Week 21.

The same template handles: maximum-sum path (replace `+` by `max` and add the cell value), minimum-cost path, and paths with an extra resource (add a third dimension, e.g. `dp[i][j][k]` = ways with exactly k turns). If moves can go in all four directions, the graph has cycles and this is no longer DP — it becomes a shortest-path problem (Weeks 16–17).

7. Complexity summary

Problem	Time	Memory
0/1 or unbounded knapsack	O(nW)	O(W) rolled
Reachable sums with bitset	O(nW/64)	O(W/64)
LIS, quadratic DP	$O(n^2)$	O(n)
LIS, tails	O(n log n)	O(n)
LCS, edit distance	O(nm)	O(nm), or O(m) if no reconstruction
Grid paths	O(nm)	O(nm), or O(m) rolled

8. Reference implementations

All programs compile with `g++ -std=c++17 -O2 -Wall` and pass the CSES/AtCoder samples.

8.1 Book Shop (0/1 knapsack, CSES 1158)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, x;
    cin >> n >> x;
    vector<int> h(n), s(n);
    for (auto &v : h) cin >> v;
    for (auto &v : s) cin >> v;
    vector<int> dp(x + 1, 0);              // dp[c] = best pages with cost <= c
    for (int i = 0; i < n; i++)
        for (int c = x; c >= h[i]; c--)    // descending: each book used once
            dp[c] = max(dp[c], dp[c - h[i]] + s[i]);
    cout << dp[x] << "\n";
}
```

Sample: `4 10 / 4 8 5 3 / 5 12 8 1` → `13`.

8.2 Increasing Subsequence (CSES 1145)

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
```

```
    vector<int> tails;   // tails[k] = smallest tail of an increasing subsequence of
length k+1
    for (int i = 0; i < n; i++) {
        int a;
        cin >> a;
        auto it = lower_bound(tails.begin(), tails.end(), a); // strict: lower_bound
        if (it == tails.end()) tails.push_back(a);
        else *it = a;
    }
    cout << tails.size() << "\n";
}
```

Sample: `8 / 7 3 5 3 6 2 9 8` → `4`.

8.3 LCS with reconstruction (AtCoder DP Contest F)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string s, t;
    cin >> s >> t;
    int n = s.size(), m = t.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++) {
            if (s[i - 1] == t[j - 1]) dp[i][j] = dp[i - 1][j - 1] + 1;
            else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
        }
    string ans;
    int i = n, j = m;
    while (i > 0 && j > 0) {
        if (s[i - 1] == t[j - 1]) { ans += s[i - 1]; i--; j--; }
        else if (dp[i - 1][j] >= dp[i][j - 1]) i--;
        else j--;
    }
    reverse(ans.begin(), ans.end());
    cout << ans << "\n";
}
```

Sample: `axyb abyxb` → `axb` (any LCS is accepted).

8.4 Edit Distance (CSES 1639)

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    string a, b;
    cin >> a >> b;
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
```

```
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = min({dp[i - 1][j] + 1,
                            dp[i][j - 1] + 1,
                            dp[i - 1][j - 1] + (a[i - 1] != b[j - 1])});
    cout << dp[n][m] << "\n";
}
```

Sample: `LOVE MOVIE` → `2`.

8.5 Grid Paths (CSES 1638)

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 1'000'000'007;
int main() {
    int n;
    cin >> n;
    vector<string> g(n);
    for (auto &row : g) cin >> row;
    vector<vector<int>> dp(n, vector<int>(n, 0));
    if (g[0][0] == '.') dp[0][0] = 1;
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++) {
            if (g[i][j] == '*') { dp[i][j] = 0; continue; }
            if (i > 0) dp[i][j] = (dp[i][j] + dp[i - 1][j]) % MOD;
            if (j > 0) dp[i][j] = (dp[i][j] + dp[i][j - 1]) % MOD;
        }
    cout << dp[n - 1][n - 1] << "\n";
}
```

Sample: the 4×4 grid from the statement → `3`.

9. Common mistakes

- **Loop direction in rolled knapsack.** Descending for 0/1, ascending for unbounded. Test on a single item with weight 1 and capacity 2: 0/1 must give `v`, unbounded `2v`.
- **Memory blow-up.** `vector<vector<int>>(1001, vector<int>(100001))` is ~400 MB. Roll the array whenever you do not need reconstruction.
- **LIS strictness.** `lower_bound` for strictly increasing, `upper_bound` for non-decreasing.
- **Treating `tails` as the LIS.** It is not, in general; use parent pointers.
- **Index shifting.** With `dp` indexed by prefix length, the current character is `s[i-1]`. Mixing conventions produces off-by-one errors that pass small samples.
- **Overflow.** Knapsack values can sum past 2^{31} (AtCoder DP D has values up to $10^9 \times 100$ items): use `long long`.

- **Forgetting the modulo** after each addition in counting DPs, or adding three terms before reducing when each can be close to MOD.
- **Trap on the starting cell** in grid problems.
- **Unreachable states in "exact" formulations.** If `dp[c]` means "exactly c", initialise with $-\infty$ (or a sentinel), not 0, or you will build answers from impossible states.

10. Practice problems

Knapsack:

- CSES 1158 — Book Shop — https://cses.fi/problemset/task/1158
- CSES 1636 — Coin Combinations II — https://cses.fi/problemset/task/1636
- CSES 1745 — Money Sums — https://cses.fi/problemset/task/1745
- CSES 1093 — Two Sets II — https://cses.fi/problemset/task/1093
- AtCoder DP Contest D — Knapsack 1 — https://atcoder.jp/contests/dp/tasks/dp_d
- AtCoder DP Contest E — Knapsack 2 — https://atcoder.jp/contests/dp/tasks/dp_e

LIS:

- CSES 1145 — Increasing Subsequence — https://cses.fi/problemset/task/1145
- Codeforces 340D — Bubble Sort Graph — https://codeforces.com/problemset/problem/340/D
- Codeforces 1350B — Orac and Models — https://codeforces.com/problemset/problem/1350/B

Two sequences:

- AtCoder DP Contest F — LCS — https://atcoder.jp/contests/dp/tasks/dp_f
- CSES 1639 — Edit Distance — https://cses.fi/problemset/task/1639

Grid and 2D state:

- CSES 1638 — Grid Paths — https://cses.fi/problemset/task/1638
- AtCoder DP Contest H — Grid 1 — https://atcoder.jp/contests/dp/tasks/dp_h
- CSES 1744 — Rectangle Cutting — https://cses.fi/problemset/task/1744

Stretch:

- CSES 1140 — Projects (DP + binary search over sorted intervals) — https://cses.fi/problemset/task/1140

11. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 7 (Dynamic programming): knapsack, LIS, edit distance, paths in a grid.
- CP-Algorithms, "Longest increasing subsequence": https://cp-algorithms.com/sequences/longest_increasing_subsequence.html
- AtCoder Educational DP Contest (https://atcoder.jp/contests/dp) — problems A–H cover everything in this lecture.
- Looking ahead: Week 21 shows that every DP is a longest/shortest path or path count in the DAG of states; Week 26 (bitmask DP) and Week 27 (interval and digit DP) add new state shapes.